ADVANCED PYTHON PROGRAMMING

UNIT IV

Advanced File Handling and Data Processing

Student Reference Guide

- Text and binary file operations; context managers
- JSON, CSV, and XML serialization
- Exception handling and reliable cleanup
- Pandas data processing, including large CSV files
- HTTP APIs with requests, status codes, and error handling

Definitions • Commands • Runnable examples • Common mistakes • Revision checklist

Contents

1. Files and file modes

2. Context managers and exceptions

3. JSON serialization

4. CSV processing

5. XML parsing

6. Pandas data analysis

7. Processing large datasets

8. HTTP APIs and requests

9. Integrated mini-project

10. Quick reference and self-assessment

1. Files and file modes

File handling is the process of creating, reading, updating, and writing data stored outside a running program. Files let data survive after a program exits.

Text files store character data interpreted using an encoding such as UTF-8. **Binary files** store bytes, such as images, PDFs, audio, and other non-text data.

The `open()` function returns a file object. Prefer an explicit encoding when working with text files.

Mode	Meaning	Important behavior
r	Read text	File must exist.
w	Write text	Creates file or truncates existing file.
a	Append text	Creates if missing; writes at the end.
x	Exclusive create	Fails if file already exists.
rb / wb	Read / write bytes	No text encoding or newline conversion.
r+	Read and write	File must exist; does not truncate automatically.

Basic text file example

```
from pathlib import Path

path = Path("notes.txt")

# "w" creates the file or replaces its contents.
with path.open("w", encoding="utf-8") as file:
    file.write("Advanced Python\n")
    file.write("File handling basics\n")

# "r" reads the existing file.
with path.open("r", encoding="utf-8") as file:
    contents = file.read()

print(contents)
```

Useful reading methods

Method	Purpose
read()	Reads the remaining content as one string (text mode).
readline()	Reads one line.
readlines()	Returns remaining lines as a list; can use substantial memory.
for line in file	Processes one line at a time; preferred for large text files.
write(text)	Writes a string and returns the number of characters written.

Appending and binary data

```
with open("log.txt", "a", encoding="utf-8") as file:
    file.write("Program started\n")

# Binary example: copy bytes from one file to another.
with open("source.bin", "rb") as source:
    with open("copy.bin", "wb") as destination:
        while chunk := source.read(8192):
            destination.write(chunk)
```

Common mistake: opening a file in `w` mode deletes its previous contents immediately. Use `a` when you intend to append.

2. Context managers and exceptions

A **context manager** sets up and reliably cleans up a resource. The `with` statement calls the context manager protocol so a file is closed even if an exception occurs.

```
with open("data.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line.rstrip())
# The file is closed after the block, including on an exception.
```

Exception-handling keywords

Keyword	Purpose
try	Contains code that may raise an exception.
except	Handles a matching exception type.
else	Runs only if the try block raised no exception.
finally	Runs whether an exception occurred or not; useful for cleanup.
raise	Raises an exception intentionally or re-raises the current one.

```
def read_score(path):
    try:
        with open(path, "r", encoding="utf-8") as file:
            score = int(file.read().strip())
    except FileNotFoundError:
        print("The score file does not exist.")
        return None
    except ValueError:
        print("The file must contain an integer.")
        return None
    else:
        print("Score loaded successfully.")
        return score
    finally:
        print("Read attempt finished.")
```

Good exception-handling practice

• Catch specific exceptions when possible; avoid a broad `except:` that hides unrelated defects.

• Do not silently ignore errors. Log them, explain them, recover safely, or re-raise.

• A `finally` block is useful for cleanup, but `with` is usually the cleanest way to manage files.

• Validate input at system boundaries: files, user input, APIs, and databases.

3. JSON serialization

JSON (JavaScript Object Notation) is a text format for exchanging structured data. Python's `json` module converts compatible Python objects to JSON and back.

Function	Use
json.dump(obj, file)	Serialize an object into an open file.
json.load(file)	Parse JSON from an open file.
json.dumps(obj)	Serialize an object into a string.
json.loads(text)	Parse JSON from a string.

```
import json

student = {
    "id": 101,
    "name": "Anu",
    "marks": [82, 91, 87],
    "active": True
}

# Write a JSON file.
with open("student.json", "w", encoding="utf-8") as file:
    json.dump(student, file, indent=4)

# Read the JSON file.
with open("student.json", "r", encoding="utf-8") as file:
    loaded = json.load(file)

print(loaded["name"])
print(loaded["marks"])

# String conversion:
text = json.dumps(student)
restored = json.loads(text)
```

JSON notes

• JSON supports objects, arrays, strings, numbers, booleans, and null. In Python these commonly map to dict, list, str, int/float, bool, and None.

• A JSON object is not identical to a Python dictionary; JSON has its own syntax and type rules.

• Functions, file handles, and arbitrary custom objects are not directly JSON-serializable without conversion.

• Handle `json.JSONDecodeError` when input may be malformed.

4. CSV processing

CSV (Comma-Separated Values) is a simple tabular text format. It is commonly used for spreadsheets, reports, and data exchange. Use the `csv` module instead of splitting lines manually because fields may contain commas, quotes, or newlines.

```
import csv

rows = [
    {"name": "Anu", "marks": 85},
    {"name": "Ravi", "marks": 92},
]

# Write rows with column headers.
```

```
with open("students.csv", "w", newline="", encoding="utf-8") as file:
    fieldnames = ["name", "marks"]
    writer = csv.DictWriter(file, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(rows)

# Read rows as dictionaries.
with open("students.csv", "r", newline="", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(row["name"], int(row["marks"]))
```

CSV tools

Tool	Purpose
csv.reader(file)	Each row is a list of strings.
csv.writer(file)	Writes lists as CSV rows.
csv.DictReader(file)	Each row is a dictionary keyed by header.
csv.DictWriter(file, fieldnames=...)	Writes dictionary rows using selected columns.
newline=''	Recommended when opening CSV files to let the csv module handle newline behavior.

Important: values read from CSV are generally strings. Convert them to numeric types before arithmetic. CSV itself does not preserve Python types.

5. XML parsing

XML (eXtensible Markup Language) stores hierarchical data using named tags and attributes. It is more verbose than JSON but appears in configuration files, enterprise integrations, and document formats.

```
# Save this as students.xml:
# <students>
#   <student id="1"><name>Anu</name><marks>85</marks></student>
#   <student id="2"><name>Ravi</name><marks>92</marks></student>
# </students>

import xml.etree.ElementTree as ET

tree = ET.parse("students.xml")
root = tree.getroot()

for student in root.findall("student"):
    student_id = student.get("id")
    name = student.findtext("name")
    marks = int(student.findtext("marks"))
    print(student_id, name, marks)
```

Security: ElementTree is not designed for untrusted XML (crafted files can exhaust memory or CPU). For such input, run `python -m pip install defusedxml` and use `from defusedxml import ElementTree as ET`.

JSON vs CSV vs XML

Format	Best fit	Structure
JSON	Nested API data, settings, objects	Objects and arrays
CSV	Flat tables, spreadsheets, datasets	Rows and columns
XML	Hierarchical documents and legacy integrations	Nested elements and attributes

6. Pandas data analysis

Pandas is a third-party Python library for labeled tabular data. A **Series** is a one-dimensional labeled array; a **DataFrame** is a two-dimensional labeled table with rows and columns.

Install and verify

```
python -m pip install pandas
python -c "import pandas as pd; print(pd.__version__)"
```

A small complete example

```
import pandas as pd

data = {
    "name": ["Anu", "Ravi", "Meena", "Imran"],
    "department": ["CSE", "ECE", "CSE", "ECE"],
    "marks": [85, 72, 95, None]
}
df = pd.DataFrame(data)

print(df.head())
print(df.shape)       # (rows, columns)
print(df.columns)
df.info()   # prints the summary itself; print() would add 'None'
print(df.describe(numeric_only=True))

# Filter rows:
high_scores = df[df["marks"] >= 80]
print(high_scores)

# Missing values:
print(df.isna().sum())
df["marks"] = df["marks"].fillna(df["marks"].mean())

# Aggregate by group:
summary = df.groupby("department")["marks"].mean()
print(summary)
```

Frequently used Pandas operations

Operation	Meaning
pd.read_csv('file.csv')	Load CSV into a DataFrame.
df.head(n)	Preview first n rows (default 5).
df.shape	Tuple: number of rows and columns.
df.columns	Column labels.
df.info()	Data types and non-null counts.
df.describe()	Summary statistics for suitable columns.
df.isna().sum()	Count missing values per column.
df.dropna()	Drop rows/columns with missing values according to options.
df.fillna(value)	Replace missing values.
df.groupby(key)[col].mean()	Calculate grouped means.
df.to_csv('out.csv', index=False)	Export without writing the index as a column.

Boolean filtering requires parentheses when combining conditions with & or |

```
# Marks at least 80 AND department is CSE
result = df[(df["marks"] >= 80) & (df["department"] == "CSE")]

# Either marks at least 90 OR department is ECE
result = df[(df["marks"] >= 90) | (df["department"] == "ECE")]
```

7. Processing large datasets

Loading an entire large file into memory can exhaust RAM. Process data in chunks when the full dataset does not need to be in memory at once.

```
import pandas as pd

total_rows = 0
total_marks = 0
count_marks = 0

for chunk in pd.read_csv("large_students.csv", chunksize=50_000):
    total_rows += len(chunk)
    valid = chunk["marks"].dropna()
    total_marks += valid.sum()
    count_marks += valid.count()

average = total_marks / count_marks if count_marks else None
print("Rows:", total_rows)
print("Average marks:", average)
```

Chunk-processing rules

• Choose a chunk size based on memory and processing cost; 50,000 is an example, not a universal best value.

• Aggregate partial results across chunks instead of concatenating all chunks back together.

• Select only needed columns with `usecols` and specify data types with `dtype` where appropriate.

• For very large or complex workloads, consider a database or a scalable processing engine instead of forcing everything into one DataFrame.

8. HTTP APIs and requests

An **API (Application Programming Interface)** defines how software components communicate. A **web API** commonly exposes resources over HTTP and often returns JSON.

HTTP method	Common intent
GET	Retrieve a resource.
POST	Create a resource or submit processing.
PUT	Replace a resource (API-dependent).
PATCH	Partially update a resource.
DELETE	Delete a resource.

Install requests

```
python -m pip install requests
```

Make a GET request safely

```
import requests

url = "https://jsonplaceholder.typicode.com/todos/1"

try:
    response = requests.get(url, timeout=(3, 10))
    response.raise_for_status()  # Raises for HTTP 4xx/5xx
    data = response.json()
    print(data)
except requests.exceptions.Timeout:
    print("The request timed out.")
except requests.exceptions.HTTPError as exc:
    print("The server returned an HTTP error:", exc)
except requests.exceptions.JSONDecodeError:   # must come before RequestException
    print("The response body was not valid JSON.")
except requests.exceptions.RequestException as exc:
    print("A network/request error occurred:", exc)
```

API concepts and error handling

Concept	Explanation
URL / endpoint	Address of a resource or operation.
Status code	HTTP result category; 2xx usually indicates success, 4xx client-side issue, 5xx server-side issue.
Headers	Metadata such as content type or authorization.
Query parameters	Values added to the URL, commonly for filtering/pagination.
Request body	Data sent with methods such as POST or PUT.
Timeout	Limits how long a request waits for connection/read activity; not necessarily a strict total wall-clock deadline.
raise_for_status()	Raises HTTPError for unsuccessful 4xx/5xx responses.

Sending JSON

```
import requests

payload = {"title": "study Python", "completed": False, "userId": 1}
response = requests.post(
    "https://jsonplaceholder.typicode.com/todos",
    json=payload,
    timeout=10
)
response.raise_for_status()
print(response.status_code)
print(response.json())
```

Security: do not hard-code API keys or passwords in source code. Use environment variables or a secrets manager. Use HTTPS for sensitive traffic and avoid logging tokens or private data.

9. Integrated mini-project: CSV summary pipeline

This small project reads a CSV file, validates numeric marks, computes a summary, and writes JSON output. Create `students.csv` in the same folder first.

```
# students.csv
# name,department,marks
# Anu,CSE,85
# Ravi,ECE,72
# Meena,CSE,95

# pipeline.py
import csv
import json
from collections import defaultdict

def build_summary(input_path, output_path):
    totals = defaultdict(float)
    counts = defaultdict(int)

    with open(input_path, "r", newline="", encoding="utf-8") as file:
        for row in csv.DictReader(file):
            try:
                department = row["department"].strip()
                marks = float(row["marks"])
            except (ValueError, KeyError, TypeError, AttributeError) as exc:
                raise ValueError(f"Invalid row: {row}") from exc

            totals[department] += marks
            counts[department] += 1

    summary = {
        dept: {
            "count": counts[dept],
            "average_marks": round(totals[dept] / counts[dept], 2)
        }
        for dept in totals
```

```
    }

    with open(output_path, "w", encoding="utf-8") as file:
        json.dump(summary, file, indent=4)

    return summary

if __name__ == "__main__":
    print(build_summary("students.csv", "summary.json"))
```

Run it

```
python pipeline.py
```

10. Unit IV quick reference and self-assessment

Need	Recommended choice
Read a text file	with open(path, encoding='utf-8')
Large text file	Iterate line by line.
Write JSON to disk	json.dump(obj, file)
Parse JSON text	json.loads(text)
CSV with headers	csv.DictReader / csv.DictWriter
Hierarchical XML	xml.etree.ElementTree
Tabular analysis	pandas DataFrame
Large CSV	pd.read_csv(..., chunksize=...)
HTTP API	requests.get(..., timeout=...).raise_for_status()

- I can explain text versus binary files and choose the correct file mode.
- I can use `with open()` and explain why cleanup matters.
- I can distinguish JSON dump/load from dumps/loads.
- I can read and write CSV dictionaries and convert string values to numbers.
- I can parse basic XML using ElementTree.
- I can inspect, filter, clean, group, and export Pandas DataFrames.
- I can process a large CSV in chunks and combine partial aggregates.
- I can call an API with a timeout, check status, parse JSON, and handle common errors.

Practice tasks

- Write a program that copies a text file while counting its lines.
- Read JSON records and print only students with marks of at least 80.
- Read a CSV and report the maximum mark per department.
- Parse an XML file and list each student's ID, name, and marks.
- Use Pandas to find missing values and export rows above a chosen threshold.
- Modify the API example to pass a query parameter and print the response status code.